

INTERPRETABLE TIME SERIES CLASSIFICATION ON A TINY BUDGET: BOTTLENECKED TCNS WITH TOP- k CONJUNCTIVE POOLING

Anonymous authors

Paper under double-blind review

ABSTRACT

Time series classifiers on sensor nodes are often asked for two things at once: a prediction, and a sign of *which part of the signal caused it*. Multiple instance learning gives both from a single forward pass, and MILLET (Early et al., 2024) is the standard method for time series. Its cost, however, comes from an InceptionTime backbone: 424 K parameters and 848 MFLOPs per series, ten to a hundred times more than the memory and compute available on microcontroller-class hardware. We ask whether *classification + interpretation* still works when the model is made ten times smaller. We replace the backbone with a length-preserving multi-scale depthwise-separable dilated convolutional encoder, and split its pointwise channel mixing — the step that holds most of its parameters — into two smaller matrices through a narrow bottleneck. We also replace the shared-gate mean of conjunctive pooling with a per-class Top- k head that averages only the κ fraction of time steps that support the class most. Neither part is new on its own; our contribution is the combination and the measurement of what it costs. The resulting model has **41.3 K parameters** and **76.7 MFLOPs** per series, $10.3\times$ and $11.1\times$ below the baseline. On WebTraffic, the only dataset here with per-time-step ground truth, it beats the equally-trained baseline at both jobs at once: accuracy 0.947 against 0.887, NDCG@ n 0.772 against 0.677. Across 85 UCR datasets it loses 1.8 accuracy points against that baseline and 3.4 against MILLET’s published recipe, and a wider version of the same encoder wins those points back — so the loss comes from the smaller size, not from the design. An ablation separates the two parts: the bottleneck removes 28 % of the parameters with no accuracy cost we can measure, and Top- k pooling gives the better interpretation. We also show that AOPCR, the faithfulness measure used in this field, changes by $5.2\times$ on a fixed architecture when only the training budget changes, so faithfulness can only be compared between models trained the same way. We report a reduction in parameters and FLOPs, not a model running on a device.

1 INTRODUCTION

A time series classifier is often not the end of the pipeline but the input to a decision. A vibration monitor that reports *bearing fault* is only useful if an engineer can see which part of the recording caused that decision; a wearable that reports *arrhythmia* can only be checked if the beats it reacted to can be inspected. Post-hoc attribution (Ribeiro et al., 2016; Lundberg & Lee, 2017; Sundararajan et al., 2017) is one way to do this, but it needs a second computation on top of the classifier, and it explains a stand-in model rather than the model itself. Rudin (2019) argues for the other way: build the explanation into the model, so that what is explained is what is predicted.

The devices that need this most are the ones least able to afford it. Condition monitoring, wearable health sensing and structural monitoring run on microcontroller-class hardware, where a common setting is a few hundred kilobytes of SRAM against about one megabyte of flash (Lin et al., 2020; Banbury et al., 2021). A model can run there only if its weights fit in flash, its activations fit in SRAM, and its computation fits in the energy available between two sensor readings. In this setting, parameter count and FLOPs per series are not extra columns in a table; they decide whether the method can be used at all.

WebTraffic #101 — true class 2: same input, 2 models — 10× fewer parameters, and it still explains itself

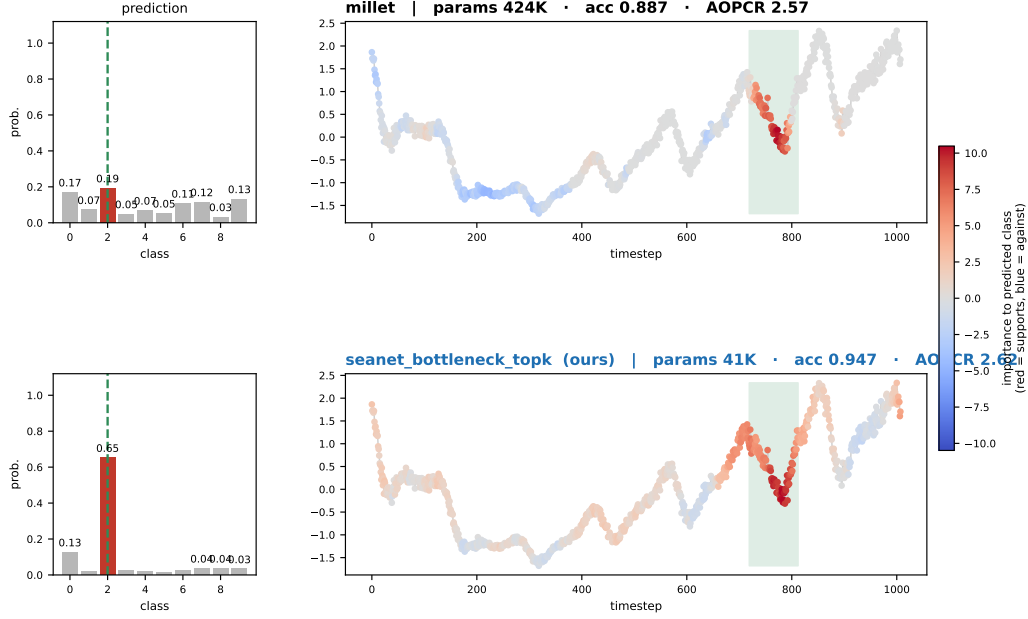


Figure 1: The same WebTraffic series through both models — *top*: the MILLET baseline (423,707 parameters), *bottom*: ours (41,324). Left, the predicted class probabilities with the true class marked; right, the series coloured by each model’s own interpretation (red supports the predicted class, blue argues against), with the shaded band showing where the generator put the class signal. Both models are correct, but the smaller one is far more sure of it — 0.65 against 0.19 on the true class — and puts its strongest evidence inside the marked window. The accuracy and AOPCR in the panel titles are dataset means over three seeds; the single series is an example, not evidence.

MILLET (Early et al., 2024) is our starting point, because it already gives both outputs we want. It treats a series as a *bag* of time steps and applies multiple instance learning (Dietterich et al., 1997; Ilse et al., 2018): an encoder produces one embedding per time step, an attention branch decides where to look, a per-step classifier decides what each position argues for, and the product of the two is both summed into the prediction and read out directly as the interpretation. The explanation is not an approximation of the model — it is a value computed inside the forward pass. MILLET is also accurate, matching or beating standard deep time series classifiers on the UCR archive (Dau et al., 2019).

The cost, however, comes from the backbone and not from the MIL part. MILLET’s reference setting uses InceptionTime (Ismail Fawaz et al., 2020): 423,707 parameters and 848 MFLOPs for one series of length 1008, so at 32-bit precision the weights alone take about 1.6 MB — more than the flash budget above, before activations, runtime or application code. Nothing in MIL requires this. Swapping MILLET’s interpretable head for ordinary global average pooling on the same backbone removes only 1,041 of those parameters (Section 3.1): the ability to explain itself costs a quarter of a percent of the model, and almost all of the cost is the encoder. This paper builds on that observation.

We therefore keep the MIL setup and change the two parts that decide its cost and how selective it is. First, the encoder: a stack of multi-scale depthwise-separable dilated convolutions, which keeps the length unchanged so that one embedding per time step reaches the head, and in which the pointwise 1×1 mixing — which holds most of the parameters once the convolutions are separable — is split into two smaller matrices through a narrow bottleneck of width $r = d/4$, halving that term. Second, the pooling head: instead of averaging the evidence over all T time steps behind one shared attention gate, we score each class separately and average only the $k = \lceil \kappa T \rceil$ largest evidence values for that class. Both parts already exist — separable convolutions (Chollet, 2017; Howard et al., 2017), squeeze-and-expand bottlenecks (He et al., 2016; Iandola et al., 2016; Sandler et al., 2018), and k -max selection in MIL (Durand et al., 2016) — and we claim neither as new. What is new is putting

them together inside an interpretable MIL time series classifier, and measuring what that combination keeps and what it gives up.

It keeps a lot and gives up something real. The resulting model uses 41,324 parameters and 76.7 MFLOPs per series, $10.3\times$ and $11.1\times$ below MILLET. On WebTraffic, the one dataset here whose generator records which time steps actually carry the class signal, it beats the equally-trained baseline at both jobs at once — accuracy 0.947 against 0.887, $\text{NDCG}@n$ 0.772 against 0.677 (Figure 1). Across 85 UCR datasets it is worse: 0.810 against 0.827 for that baseline and 0.843 for MILLET’s published recipe. We show where that loss comes from instead of hiding it in an average: a wide version of the same encoder, at 269 K parameters, reaches 0.829 and ranks above the equally-trained baseline, so the price is paid for the smaller size and not for a flaw in the design.

Contributions.

- **A MIL encoder small enough to be a candidate for small devices** (Section 3.2): multi-scale depthwise-separable dilated convolutions with a bottlenecked pointwise mix, keeping the length unchanged so that the per-time-step interpretation is not lost, at $10.3\times$ fewer parameters and $11.1\times$ fewer FLOPs than MILLET.
- **Per-class Top- k conjunctive pooling** (Section 3.3), which adds no parameters and becomes *exactly* the baseline head at $\kappa = 1$ (Property 1) — the property that makes the κ study of Section 5.4 a controlled experiment rather than a comparison of two different models.
- **An ablation** separating the two changes (Section 5.4): the bottleneck buys the size reduction with no accuracy cost we can measure, and Top- k buys the interpretation quality, measured against per-time-step ground truth.
- **A warning about the AOPCR measure** (Section 5.5): AOPCR has no fixed scale, and with the architecture and the code held fixed it changes by $5.2\times$ when only the training budget changes, so we compare only against baselines we retrained ourselves.

We do not run the model on a microcontroller and report no on-device latency, energy or memory: our cost evidence is parameter counts, FLOPs counted by hand from the layer shapes, and GPU timings. This shows the model *could fit* such a budget, not that we ran it there. We also do not claim state of the art on UCR; Section 5.3 measures what we give up instead of playing it down.

2 RELATED WORK

Deep time series classification. Convolutional models have been the strongest in this field since FCN and ResNet were shown to work well without hand-made features (Wang et al., 2017), and InceptionTime (Ismail Fawaz et al., 2020) is still the reference deep architecture on the UCR archive (Dau et al., 2019); the survey of Ismail Fawaz et al. (2019) covers the designs between them. Accuracy in this field is almost always reported without a cost column, and the models are large: those three architectures use between 267 K and 506 K parameters at the input length we study.

Interpretability and multiple instance learning. Attribution methods (Ribeiro et al., 2016; Lundberg & Lee, 2017; Sundararajan et al., 2017) explain a trained model afterwards; Rudin (2019) argues for building the explanation in instead, and for time series VQShape (Wen et al., 2024) and InterpGN (Wen et al., 2025) do so with shapelet-like tokens and gated expert models. MIL (Dietterich et al., 1997) gives a different way, in which the interpretation is a value computed inside the network rather than a separate output. Attention-based MIL (Ilse et al., 2018) and its dual-stream version (Li et al., 2021) developed the pooling methods for histopathology, and MILLET (Early et al., 2024) moved the method to time series with *conjunctive pooling*, the baseline we start from. Keeping only the strongest instances instead of averaging all of them is also not new: WELDON (Durand et al., 2016) introduced k -max region selection for weakly supervised recognition, and the same idea appears again in weakly supervised segmentation.

Small architectures and deployment budgets. Depthwise-separable convolution (Chollet, 2017; Howard et al., 2017) and squeeze-and-expand bottlenecks (He et al., 2016; Iandola et al., 2016; Sandler et al., 2018) are the two standard ways to reduce the parameter count of a convolutional network, and both carry over directly to the 1-D case. For time series, ATCN (Baharani & Tabkhi,

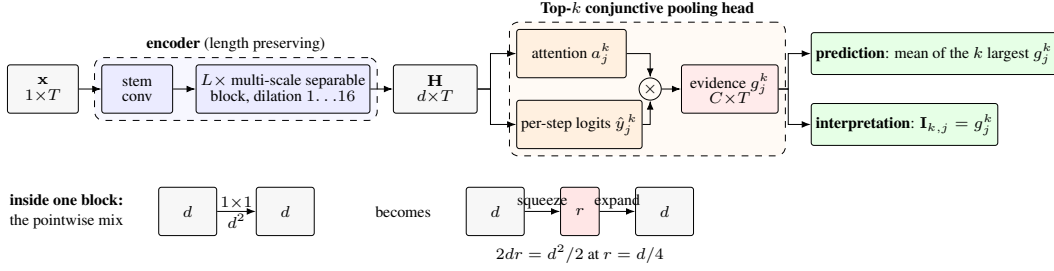


Figure 2: *Top*: the model. The encoder never changes the length, so everything after it keeps a time axis. The head’s two branches multiply into the evidence g_j^k , the single quantity from which *both* outputs are read: the prediction is a sum of it over time, and the interpretation *is* it. *Bottom*: the change inside the block — the pointwise 1×1 channel mix is split into two smaller matrices through a narrow middle layer, halving the term that holds most of the parameters.

2022) combines residual connections with separable convolution for embedded inference, and Risso et al. (2023) search the dilation and channel structure of TCNs under edge constraints; neither aims at interpretability — they produce a label, and where it came from is left open. On the deployment side, MCUNet (Lin et al., 2020) and MicroNets (Banbury et al., 2021) describe what fits on common microcontrollers (flash for weights, SRAM for activations, operation count as a stand-in for latency) and both find that architectures designed without those limits do not simply shrink into them. Post-training integer quantisation (Jacob et al., 2018) is the standard last step and cuts weight memory by about four times. We use these budgets as the target the paper aims at, not as a result: no measurement here was taken on a microcontroller.

Our contribution. Every part we use already exists. What is missing above is the combination: small time series architectures do not produce interpretations, interpretable ones are not built to a cost budget, and work on MIL pooling judges selection by bag-level accuracy rather than against per-instance ground truth. This paper fills that gap. We contribute the per-class form of k -max selection inside conjunctive pooling, with k set as a fraction of the series length and with the exact return to the baseline head at $\kappa = 1$ (Property 1); its pairing with a bottlenecked encoder that keeps the length unchanged; an ablation that assigns the size saving and the interpretation quality to the two parts separately; and the finding that the faithfulness measure used in this field cannot be compared across training budgets.

3 METHOD

3.1 BACKGROUND: MULTIPLE INSTANCE LEARNING FOR TIME SERIES

A series $\mathbf{X} \in \mathbb{R}^{1 \times T}$ is treated as a bag of T instances, one per time step. An encoder f produces $\mathbf{H} = f(\mathbf{X}) \in \mathbb{R}^{d \times T}$, one embedding $\mathbf{z}_j$ per step. MILLET’s *conjunctive pooling* then runs two branches over those embeddings and multiplies them:

$$a_j = \frac{\exp(w^\top \tanh(V\mathbf{z}_j))}{\sum_i \exp(w^\top \tanh(V\mathbf{z}_i))}, \quad \mathbf{y}_j = W_c \mathbf{z}_j \in \mathbb{R}^C, \quad g_j^k = a_j \mathbf{y}_j^k, \quad (1)$$

where a_j says how much step j matters and $\mathbf{y}_j^k$ says what step j argues for regarding class k . The product g_j^k is the *evidence* of step j for class k , and both outputs are read from it:

$$\hat{y}^k = \frac{1}{T} \sum_{j=1}^T g_j^k \quad (\text{prediction}), \quad \mathbf{I}_{k,j} = g_j^k \quad (\text{interpretation}). \quad (2)$$

This is why the explanation is faithful by construction: it is not a model of the model, it is one of the terms the prediction adds up. Three properties of Equations (1) and (2) matter later. **(L1)** the gate a_j is shared across classes, so every class is forced to look in the same place; **(L2)** the values a_j are normalised over time, so they share one fixed total instead of being absolute; and **(L3)** the prediction is a fixed mean over all T steps, so a signal that sits in a short window is divided by the length of the whole series.

Where the cost is. The MIL part in Equation (1) is almost free. Replacing MILLET’s interpretable head with ordinary global average pooling on the same InceptionTime backbone changes the model from 423,707 to 422,666 parameters: explaining itself costs 0.25 % of the model, and everything else is the encoder. Making an interpretable classifier smaller is therefore an *encoder* problem, not a pooling problem — which is what Section 3.2 works on, while Section 3.3 works on quality at a fixed cost.

3.2 SEPARABLE ENCODER WITH A BOTTLENECK

We replace InceptionTime with a stack of *multi-scale separable residual blocks*. Each unit reads its input with three depthwise convolutions of kernel size $\{5, 11, 23\}$ at a shared dilation, sums them, and mixes channels with a pointwise 1×1 convolution, followed by BatchNorm (Ioffe & Szegedy, 2015), ReLU and dropout; two units and a residual connection form a block, and dilation doubles from block to block up to a cap of 16. Two design choices are forced by the interpretation rather than by accuracy: the convolutions use “same” padding with no stride and no pooling, so T stays the same at every stage — needed, because Equation (2) needs one value per input step and the faithfulness metrics change individual steps — and the dilation cap keeps every embedding tied to a limited window, so importance stays local and easy to read. The third choice is about cost: the depthwise/pointwise split (Chollet, 2017; Howard et al., 2017) replaces the $d^2 k$ weights of a dense convolution with

$$\underbrace{dk}_{\text{depthwise}} + \underbrace{d^2}_{\text{pointwise}} \ll d^2 k, \quad (3)$$

which is what makes a model this small possible at all: at $d = 128$ a dense convolution covering the same three kernel sizes would cost 638,976 weights per step against 22,144 for the separable unit.

The bottleneck. Equation (3) has a consequence: once the convolutions are separable, the *pointwise* term is the largest cost. At $d = 128$ the three depthwise convolutions of one unit cost 5,376 weights while the single 1×1 costs 16,512 — three times all three depthwise convolutions together. We therefore split that 1×1 into two smaller matrices with a narrow middle layer of width $r = d/4$, squeezing and then expanding:

$$P = P_{\text{expand}} P_{\text{squeeze}}, \quad P_{\text{squeeze}} \in \mathbb{R}^{r \times d}, \quad P_{\text{expand}} \in \mathbb{R}^{d \times r}, \quad \text{cost } d^2 \rightarrow 2dr = d^2/2. \quad (4)$$

This is the standard squeeze-and-expand bottleneck (He et al., 2016; Iandola et al., 2016; Sandler et al., 2018); what matters here is where we apply it and what it does not break. It halves the biggest term for any d , and the block still takes in and returns (d, T) , so the length is still unchanged and the per-step interpretation still works. At the setting used throughout ($d = 64$, 4 blocks) it takes the whole model from 57,580 to 41,324 parameters and from 109.7 to 76.7 MFLOPs per series: 28 % of the parameters and 30 % of the computation, for a change the head never sees.

3.3 TOP- k CONJUNCTIVE POOLING

The head is where we fix limitations L1 and L3, and it costs no extra parameters. First we give each class its own gate, $a_j \rightarrow a_j^k$, so that classes may disagree about where to look (L1). Then, instead of the fixed mean of Equation (2), we average only the largest evidence values for each class, with the count set as a fraction κ of the series length so that the head works for any length:

$$k = \max(1, \lceil \kappa T \rceil), \quad \hat{y}^k = \frac{1}{k} \sum_{j \in S_{\text{top}}^k} g_j^k, \quad \mathbf{I}_{k,j} = g_j^k, \quad (5)$$

where S_{top}^k holds the indices of the k largest g_j^k . Keeping only the strongest instances is not new in MIL (Durand et al., 2016); the form here is per-class, written as a fraction of the length, and applied to conjunctive evidence, which gives it a property that the κ experiment in Section 5.4 depends on.

Property 1 (Exact reduction). *At $\kappa = 1$, Equation (5) is class-wise conjunctive pooling term for term. For $\kappa < 1$ the gradient reaches only the selected steps.*

Two things follow. In practice, κ is a hyperparameter and not a weight, so this head has *exactly* the same parameter count as the head it extends — the better interpretation it gives is free in every cost

column of Table 2. In experiments, the $\kappa = 1$ row of Table 3 is not a different model but the same model with the selection switched off, so any difference along that column is caused by the selection and by nothing else.

Training objective. Every model is trained with cross-entropy on the bag logits plus one term that exists for the explanation rather than for the classification:

$$\mathcal{L} = \text{CE}(\hat{y}, y) + \lambda_{\text{focus}} \left(- \sum_j \bar{a}_j \log(\bar{a}_j + \epsilon) \right), \quad (6)$$

with $\bar{a}_j$ the attention averaged over classes. Attention spread evenly over a series has high entropy, so adding entropy to a loss we minimise pushes the model towards attention that is concentrated in fewer places. Without this term a model can be right while its importance map is flat, and then a reader cannot tell where the evidence stops. We set $\lambda_{\text{focus}} = 0.01$ for our models and 0 for the MILLET baseline, since the term is our addition; Section 5.4 reports what it is worth.

4 EXPERIMENTAL SETUP

Research questions. **RQ1** Does the small model retain classification accuracy? **RQ2** Does it retain a useful interpretation? **RQ3** How much cost is actually removed? **RQ4** What does the bottleneck contribute? **RQ5** What does Top- k pooling contribute?

Datasets. WebTraffic is the synthetic benchmark released with MILLET (Early et al., 2024): 1,000 series (500 train / 500 test) of length 1,008 over 10 classes, in which a generator puts the class signal inside a known window. It is the only dataset here with *per-time-step ground truth*, so it is the only place where we can score an interpretation instead of just looking at it, and RQ2 is answered there. For wider coverage we use the UCR archive (Dau et al., 2019): we report the 85-dataset subset used by MILLET, so that the numbers can be compared, and the full 128-dataset archive in Section C.2.

Metrics. Classification is test accuracy, and across UCR also mean rank over the datasets that every model in the comparison finished (Demšar, 2006), reported in Section C.2. Interpretation is measured in two ways. **AOPCR** (Samek et al., 2017) removes time steps in the order the model says they are important and adds up the drop in the score of the predicted class; it needs no ground truth, so we can use it everywhere, but it has no fixed scale — a point Section 5.5 comes back to. **NDCG@ n** (Järvelin & Kekäläinen, 2002) ranks the time steps by the importance the model claims and scores that ranking against the true signal window from the generator; it lies in $[0, 1]$ and needs ground truth, so it exists only on WebTraffic. NDCG@ n is the stronger of the two and we treat it that way.

Baselines. MILLET with its reference InceptionTime backbone (Ismail Fawaz et al., 2020) is the model this paper is about; FCN and ResNet (Wang et al., 2017) with the same head give two more reference points. Because Section 5.5 shows that faithfulness numbers cannot be compared between different training recipes, we retrain every baseline ourselves instead of copying published values, and we report MILLET twice: once under *our* budget, which is the only fair comparison, and once under *its own* published 1,500-epoch recipe, which is the harder target.

Training setup. One setting is used for every model and every dataset, so that a difference between two rows is a difference between two models: Adam (Kingma & Ba, 2015) at learning rate 1.25×10^{-3} , weight decay 1.2×10^{-4} , at most 400 epochs with early stopping at patience 60, batch size $\min(\lfloor n_{\text{train}}/10 \rfloor, 16)$ floored at 2, label smoothing 0.13, stratified 80/20 validation split when $n_{\text{train}} \geq 100$. Encoders use $d = 64$ with 4 blocks unless marked *wide* ($d = 128$, 6 blocks), and $\kappa = 0.1$; the rest of the settings are in Section B.3. Cost is measured separately at batch 32, length 1008, 10 classes on one GPU: parameters counted directly, FLOPs counted by hand from the layer shapes for one series with one multiply-add counted as two operations, and latency as the mean of 30 timed passes after 30 warm-up passes that we throw away.

Two limitations to keep in mind. First, WebTraffic was also the set we used to choose the design, so WebTraffic results are optimistic by design; UCR was never used to pick anything. Second, the seed counts are uneven: three models were run with three seeds (Table 1, column S) and the rest with one, because we ran 72 encoder-head combinations over up to 129 datasets with a fixed amount of

Table 1: Main comparison. WebTraffic has per-time-step ground truth; UCR-85 is the wider check and was never used to choose a model. S = seeds; rows with more than one seed are means. Cost is at batch 32, length 1008, 10 classes. ‡ 84 of 85 datasets finished. † MILLET’s own 1,500-epoch recipe: a harder accuracy target, and an AOPCR value that *cannot* be compared with the rest of that column (Section 5.5). Bold marks the best matched-budget row in each column.

Model	S	WebTraffic			UCR-85	Cost	
		Acc. $\uparrow$	AOPCR $\uparrow$	NDCG@ n $\uparrow$	Acc. $\uparrow$	Params	MFLOPs
FCN + conjunctive	1	0.742	3.826	0.533	0.814	267,035	535.2
ResNet + conjunctive	1	0.772	2.952	0.554	0.815	506,331	1013.2
MILLET (matched budget)	3	0.887	2.569	0.677	0.827‡	423,707	847.6
MILLET (published recipe) †	1	0.920	13.268	0.661	0.843 ‡	423,707	847.6
SEA-Net wide + class-wise	1	0.946	1.722	0.686	0.829	269,164	523.7
SEA-Net gated + Top- k	3	0.955	2.225	0.750	0.808	61,740	109.7
SEA-Net bottleneck + Top-k (ours)	3	0.947	2.621	0.772	0.810	41,324	76.7

Table 2: Cost, measured at batch 32, length 1008. Weight memory is *calculated* from the parameter count (4 B and 1 B per weight); nothing was quantised or run on a device. Latency and peak memory are GPU measurements, not estimates for a device.

Model	Params	Weights (KB)		MFLOPs	Lat. (ms)	Mem. (MB)
		fp32	int8			
MILLET	423,707	1655	414	847.6	0.203	112.3
SEA-Net wide	269,164	1051	263	523.7	0.359	123.7
SEA-Net gated	61,740	241	60	109.7	0.128	64.9
SEA-Net bottl.	41,324	161	40	76.7	0.131	179.0
SEA-Net bottl., 2 blocks	21,548	84	21	40.0	0.069	178.9

compute. The three-seed standard deviations of our main model are ± 0.016 for accuracy, ± 0.214 for AOPCR and ± 0.013 for NDCG@ n (Section C.1), and anywhere in this paper we treat a gap smaller than those as telling us nothing.

5 RESULTS

5.1 CLASSIFICATION AND INTERPRETATION ON WEBTRAFFIC

Table 1 answers RQ1 and RQ2 on WebTraffic, and both point the same way. Against the baseline trained under our budget, the 41 K model is 6.0 accuracy points better (0.947 against 0.887, three seeds each, seed standard deviation ± 0.016) and 9.5 NDCG@ n points better (0.772 against 0.677, ± 0.013); both gaps are several times the seed spread. AOPCR is 2.621 against 2.569, which is inside the AOPCR standard deviation of either model (± 0.214 and ± 0.884), so it should be read as a tie. In short: better classification, better interpretation ranking, and no change in faithfulness — with $10.3\times$ fewer parameters.

Two rows put that in context. MILLET under its own 1,500-epoch recipe reaches 0.920, so more than half of our gain over the matched-budget baseline comes from the training budget and not from the architecture; our model is still ahead of it by 2.7 accuracy and 11.1 NDCG@ n points at $10.3\times$ less cost. And SEA-Net gated + Top- k , at 61,740 parameters, is on paper the most accurate row at 0.955 — but 0.008 against a seed spread of 0.016 is not a difference we claim, and we prefer the bottleneck version because it is 33% smaller with no loss on NDCG@ n .

5.2 MODEL COST

Table 2 separates what improved from what did not. Parameters fall $10.3\times$ and computation $11.1\times$; at 32-bit precision the weights go from about 1.6 MB to 161 KB, which moves the model from above to well below the roughly 1 MB flash budget of the microcontroller class described by Lin et al.

Table 3: Ablations on WebTraffic, all at $d = 64$, 4 blocks, seed 0. *Left*: the two parts on and off — reading down gives the bottleneck, reading across gives Top- k . *Right*: only κ changes, and at $\kappa = 1$ the head is the class-wise baseline (Property 1); the last row instead turns off the entropy term of Equation (6).

Encoder	Head	Params	Acc.	AOPCR	NDCG@ n
plain	class-wise	57,580	0.916	2.478	0.719
plain	Top- k	57,580	0.932	2.691	0.778
bottleneck	class-wise	41,324	0.902	2.572	0.733
bottleneck	Top- k	41,324	0.938	2.778	0.777

κ	Acc.	AOPCR	NDCG@ n
0.05	0.888	2.288	0.715
0.10	0.938	2.778	0.777
0.25	0.940	2.648	0.733
0.50	0.882	3.114	0.732
1.00 (= baseline head)	0.908	2.436	0.722
$\lambda_{\text{focus}} = 0$	0.950	2.303	0.765

(2020) and Banbury et al. (2021), and standard int8 quantisation (Jacob et al., 2018) would take it to about 40 KB. That is the sense in which it becomes a candidate for such a device.

The other two columns do not improve, and we report them instead of leaving them out. Latency improves by only $1.6\times$ even though the FLOPs fall $11.1\times$, because depthwise convolutions are limited by memory speed and a GPU at batch 32 is the wrong tool for a claim about small hardware — the 269 K wide model is in fact the *slowest* row. Peak memory use is 179.0 MB against the baseline’s 112.3 MB, which is worse, and it is worse because of the design: the encoder keeps the length unchanged, so the activations are (d, T) at every stage no matter how few weights produce them, and Equation (4) reduces weights, not activations. On microcontrollers the SRAM needed for activations is the limit at least as often as the flash needed for weights, so this is a real limitation of the design and not a measurement error (Section 6).

5.3 ACCURACY ON THE UCR ARCHIVE

UCR was never used to choose anything, and it is where the trade-off shows. Our 41 K model reaches 0.810 mean accuracy against 0.827 for the matched-budget baseline and 0.843 for MILLET’s published recipe: 1.8 and 3.4 points behind, with the same order in mean rank over the 84 shared datasets, 15.40 against 12.60 and 9.41 (Section C.2).

Where that loss comes from matters more than its size, and one row of Table 1 answers it. The *wide* version of the same encoder — same blocks and same kind of head, with d raised from 64 to 128 and 4 blocks to 6 — reaches 0.829 and mean rank 11.53, ahead of the matched-budget baseline on both. So the architecture is not what loses on UCR; the smaller size is. Two comparisons follow, and they are the useful numbers here: changing the backbone at a similar size is worth about 1.1 rank places, while going from our 400-epoch budget to MILLET’s 1,500-epoch budget on a *fixed* architecture is worth about 3.2 — three times more.

5.4 ABLATION STUDY

RQ4, the bottleneck. Reading down Table 3, it removes 16,256 parameters — 28% of the model — and accuracy moves by -0.014 with the class-wise head and $+0.006$ with Top- k , both inside the ± 0.016 seed spread; NDCG@ n moves by $+0.014$ and -0.001 . So the safe statement is that the bottleneck buys the size reduction *with no accuracy cost we can measure*, not that it improves anything: the rest of the model does not notice the change.

RQ5, Top- k pooling. Reading across, NDCG@ n rises by 0.059 on the plain encoder and 0.044 on the bottleneck encoder. Against a NDCG@ n seed spread of ± 0.013 these are three to four standard deviations, and they are the claim we make for the head; the accuracy changes ($+0.016$ and $+0.036$) point the same way but are at or below one seed spread, so we do not present Top- k as an accuracy improvement. The right panel of Table 3 tests the selection on its own, and here Property 1 earns its place: the $\kappa = 1$ row is exactly the baseline head, so the 0.777 against 0.722 gap between $\kappa = 0.1$ and $\kappa = 1$ is caused by the selection alone. Two things in that table are worth saying plainly instead of hiding. There is a *best value*, not a steady gain — $\kappa = 0.05$ is worse than $\kappa = 0.10$ in every column, so keeping too few steps is as harmful as keeping all of them, and κ has to be chosen. And AOPCR does not follow κ at all: it is highest at $\kappa = 0.50$, where NDCG@ n is near its lowest. The last row shows that the attention-entropy term is a second, separate control that works in the same

direction: turning it off improves accuracy (0.950) but costs $\text{NDCG}@n$ (0.765) and AOPCR (2.303). Every row is a single seed, so we read the table for the overall pattern and not for single values.

5.5 SENSITIVITY OF AOPCR TO THE TRAINING BUDGET

One number in Table 1 needs an explanation, and the explanation applies more widely. MILLET under its published recipe scores AOPCR 13.268 on WebTraffic, five times every other row, with the same architecture, code, metric implementation and data as the matched-budget MILLET row, which scores 2.569; only the training recipe is different. On UCR-85 the same pair gives 0.812 against 3.926, a factor of 4.84. The reason is that AOPCR adds up a drop in the logit that has *no fixed scale* (Section A): a model trained longer and to a lower loss has sharper logits, so changing the input produces a larger absolute drop whether or not the explanation points at the right place. In line with this, the same pair moves by only 1.6 points of $\text{NDCG}@n$ — the measure that does have a fixed scale — and moves *down*, 0.677 to 0.661, while AOPCR is multiplied by 5.2. So an AOPCR value from one paper cannot be compared with one from another: it should only be reported between models trained the same way, or replaced by a measure with a fixed scale wherever ground truth allows. That is why we retrain every baseline instead of copying published numbers, and never compare against MILLET’s published AOPCR — including where it would make our results look better.

6 DISCUSSION AND LIMITATIONS

The trade-off. Section 5 in one sentence: ten times fewer parameters and ten times less computation, in exchange for about two accuracy points on a large archive, with the interpretation better rather than worse where we can score it. Whether that is a good trade depends on where the model runs, not on the model itself. On a workstation it is a bad trade. On a sensor node with a megabyte of flash it is the difference between a method that can be used and one that cannot, and the interpretation, which is the reason to use MIL at all, is not lost. Since the wide encoder wins back the UCR points (Section 5.3), the price is paid for the smaller size, and it can be changed by choosing a different width. The wider point: the tension between “interpretable” and “small” was never part of MIL — its head costs 0.25 % of the baseline — but came from a backbone chosen when cost was not a design choice.

Limitations. **No deployment:** we report parameter counts, FLOPs counted by hand from the layer shapes and GPU timings; no model was quantised, compiled or run on a microcontroller, and the int8 column of Table 2 is a calculation, not a measurement. **Activation memory did not shrink:** peak memory is worse than the baseline’s (Section 5.2) because the encoder keeps the length unchanged by design, so where SRAM and not flash is the limit, this is the wrong design; versions that stream the input or use strides while keeping a per-step interpretation are the obvious next step. **Interpretation is scored on one dataset:** $\text{NDCG}@n$ needs per-time-step ground truth, which only the synthetic WebTraffic generator gives, and that is also the set we chose the design on — so the interpretation claim rests on one optimistic benchmark, with AOPCR as the weaker backup (Section 5.5). **Uneven seeds:** three models have three seeds and the ablations use one seed each, read as a pattern against the spreads in Section 4; running all 72 combinations with several seeds needed more compute than we had. **Training is slower:** 117.3 ± 14.0 s to converge on WebTraffic against 50.4 ± 9.5 s for the baseline, because GPU kernels are tuned for dense convolutions and not for depthwise ones. The saving is at inference time, where the target device spends its life, but the training cost is real.

7 CONCLUSION

Multiple instance learning gives a time series classifier a prediction and a per-time-step explanation from one forward pass, but the standard version carries a backbone ten to a hundred times larger than what fits on the devices that need both the most. We showed that the cost is the encoder and not the interpretability, and that a separable dilated encoder with a bottleneck and per-class Top- k pooling gives a 41 K-parameter, 76.7 MFLOP model that beats the equally-trained baseline at both jobs where ground truth exists, and stays two accuracy points behind it across 85 UCR datasets. Built-in interpretability is not what makes these models large, and it does not have to be given up to make them small.

REPRODUCIBILITY STATEMENT

Everything needed to reproduce the results is either in the paper or in the anonymous code release. The models are defined in Section 3 and, for the encoder and pooling variants not used in the main text, in Sections B.1 and B.2. The single training setting used for every model and every dataset is listed in Section 4 and repeated with all the remaining hyperparameters in Section B.3. The two evaluation metrics are defined in Section 4, and the perturbation procedure behind AOPCR, which Section 5.5 shows depends on the training recipe, is given in Section A. WebTraffic is produced by the released generator of Early et al. (2024) at the settings in Section B.3; the UCR archive (Dau et al., 2019) is public and used with its published train/test splits and no preprocessing other than standardising each series. Seed counts are given per row in Table 1 and the per-seed values are in Section C.1. Cost figures come from the measurement procedure in Section 4; the script that produces them is in the code release. All 72 model combinations are listed row by row in Section C.4. Anonymous code, configurations and result tables: <https://anonymous.4open.science/r/seanet-ANON>.

ETHICS STATEMENT

This work collects no new data and involves no human subjects. It uses the public UCR archive (Dau et al., 2019) under its original terms and a synthetic benchmark produced by released code; some UCR datasets come from human physiological recordings, but they are distributed in anonymised form by their maintainers and we did not try to re-identify them or add to them. The one risk worth naming is specific to this topic: a model that shows an explanation makes people trust it more than one that does not, and Section 5.5 shows that a common faithfulness score can be raised by training longer rather than by explaining better. Interpretations from models of this kind should therefore be checked on the target domain before they are used to support decisions about people. The code we build on is released under the Apache 2.0 licence and its attribution notice is kept in our release.

REFERENCES

- Mohammadreza Baharani and Hamed Tabkhi. ATCN: Resource-efficient processing of time series on edge. *ACM Transactions on Embedded Computing Systems*, 21(5):1–21, 2022.
- Colby Banbury, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul Whatmough. MicroNets: Neural network architectures for deploying TinyML applications on commodity microcontrollers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- François Chollet. Xception: Deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1251–1258, 2017.
- Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.
- Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1-2):31–71, 1997.
- Thibaut Durand, Nicolas Thome, and Matthieu Cord. WELDON: Weakly supervised learning of deep convolutional neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4743–4752, 2016.
- Joseph Early, Gavin K. C. Cheung, Kurt Cutajar, Hanting Xie, Jasmina Kandola, and Niall Twomey. Inherently interpretable time series classification via multiple instance learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

- Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size. *arXiv preprint arXiv:1602.07360*, 2016.
- Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *International Conference on Machine Learning (ICML)*, pp. 2127–2136, 2018.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.
- Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: A review. *Data Mining and Knowledge Discovery*, 33(4):917–963, 2019.
- Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. Inceptiontime: Finding AlexNet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962, 2020.
- Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.
- Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Bin Li, Yin Li, and Kevin W. Eliceiri. Dual-stream multiple instance learning network for whole slide image classification with self-supervised contrastive learning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14318–14328, 2021.
- Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. MCUNet: Tiny deep learning on IoT devices. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “why should I trust you?”: Explaining the predictions of any classifier. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.
- Matteo Risso, Alessio Burrello, Francesco Conti, Lorenzo Lamberti, Yukai Chen, Luca Benini, Enrico Macii, Massimo Poncino, and Daniele Jahier Pagliari. Lightweight neural architecture search for temporal convolutional networks at the edge. *IEEE Transactions on Computers*, 72(3): 744–758, 2023.
- Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.
- Wojciech Samek, Alexander Binder, Grégoire Montavon, Sebastian Lapuschkin, and Klaus-Robert Müller. Evaluating the visualization of what a deep neural network has learned. *IEEE Transactions on Neural Networks and Learning Systems*, 28(11):2660–2673, 2017.
- Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, pp. 3319–3328, 2017.

Zhiguang Wang, Weizhong Yan, and Tim Oates. Time series classification from scratch with deep neural networks: A strong baseline. In *International Joint Conference on Neural Networks (IJCNN)*, pp. 1578–1585, 2017.

Yunshi Wen, Tengfei Ma, Tsui-Wei Weng, Lam M. Nguyen, and Anak Agung Julius. Abstracted shapes as tokens – a generalizable and interpretable model for time-series classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://openreview.net/forum?id=pwKkNSuuEs>.

Yunshi Wen, Tengfei Ma, Ronny Luss, Debarun Bhattacharjya, Achille Fokoue, and Anak Agung Julius. Shedding light on time series classification using interpretability gated networks. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=n34taxF0TC>.

A METRIC DEFINITIONS

AOPCR. Let $\hat{y}^c(\mathbf{X})$ be the logit of the predicted class c on the unchanged series, and let $\mathbf{X}^{(m)}$ be the series with the m time steps that the model itself ranks most important for c replaced by the mean of that series. AOPCR adds up the drop over this order,

$$\text{AOPCR} = \frac{1}{M+1} \sum_{m=0}^M \left(\hat{y}^c(\mathbf{X}) - \hat{y}^c(\mathbf{X}^{(m)}) \right), \quad (7)$$

following Samek et al. (2017) as adapted to time series by Early et al. (2024), with M set to 10 % of the series length in steps of 1 %. Higher is better. Note that Equation (7) is a difference of logits, and logits have *no fixed scale*. This is the cause of the comparison problem described in Section 5.5: two models whose logits sit on different scales give AOPCR values on different scales, however good or bad their rankings are.

NDCG@ n . Where per-time-step ground truth exists, we instead rank the T time steps by the importance $\mathbf{I}_{c,j}$ the model claims, and score that ranking with normalised discounted cumulative gain (Järvelin & Kekäläinen, 2002) at cut-off n , the number of steps the generator actually marked as carrying the class signal. A step is either relevant or not. The score is divided by the score of the best possible ranking of the same series, so NDCG@ n lies in $[0, 1]$ and can be compared between models, datasets and papers — which is why the interpretation claims of this paper rest on it and not on AOPCR.

B MODEL AND TRAINING DETAILS

B.1 ENCODER VARIANTS

The main text uses two encoders: the plain multi-scale separable stack of Section 3.2 (`seamstcn_sep`) and its bottleneck form (Equation (4), `seamstcn_sep_bottleneck`). We also ran five other variants, shown in Table 4: *gated*, which turns the time axis into one summary vector and uses it to re-weight the channels; *input-gated*, which reads the same gate from the raw input instead; *spike/trend*, which runs a smoothed branch and a high-pass branch and mixes them; *reconstruction-residual*, which adds features built from a reconstruction error; and two *wrappers* that do not replace an encoder but sit around one — *multi-scale channels*, which adds rolling max/min/std input channels, and *multi-scale pyramid*, which runs the same encoder at four downsampling rates and joins them back to length T . All of them keep T unchanged. Only the bottleneck is part of this paper’s contribution; the others are background showing that the bottleneck was the right place to cut.

B.2 POOLING HEADS

Every head keeps the two-branch form of Equation (1), so a head is fully described by two choices: the shape of the attention branch, and the operation that turns the T evidence values g_j^k into one

Table 4: Encoders: the mean over every head the encoder was paired with, WebTraffic. n is the number of pairings. These are means over different pairings, so they show the spread, not a ranking of encoders.

Encoder	Acc.	AOPCR	NDCG@ n	n
multi-scale channels (wrapper)	0.937	2.385	0.747	4
reconstruction-residual	0.924	2.315	0.719	5
input-gated	0.904	2.345	0.718	5
gated	0.902	2.056	0.710	19
plain (sea_mstcn_sep)	0.898	1.933	0.694	12
bottleneck	0.884	2.209	0.694	8
spike/trend	0.858	1.814	0.677	7
multi-scale pyramid (wrapper)	0.771	1.394	0.604	3

number. Table 5 lists all seven by those two choices, together with the setting at which each one becomes conjunctive pooling exactly. Two come from histopathology MIL rather than from us: gated attention is due to Ilse et al. (2018) and the dual-stream mechanism to Li et al. (2021); our changes to those two are the move to time series, the per-class form, and — for the dual stream — dropping its contrastive pre-training and adding a learnable mix.

Table 5: The seven heads as two separate choices. “Reduces at” gives the setting at which the head becomes conjunctive pooling exactly. L1–L3 are the limitations listed in Section 3.1.

Head	Attention	Reduction over time	Reduces at	Targets
Conjunctive (baseline)	shared gate	mean	—	—
Class-wise conjunctive	per-class gate	mean	$a_j^k = a_j$	L1
Softmax conjunctive	per-class score	softmax, learnable τ	$\tau \rightarrow \infty$	L2
Adaptive class-wise	per-class gate	soft-argmax, learnable β_k	$\beta_k \rightarrow 0$	L3
Top- k conjunctive	per-class gate	mean of the k largest	$\kappa = 1$	L3
Attention-max	per-class gate	blend of mean and max	$\lambda_k \rightarrow 0$	L3
Per-class gated attention	gated $\tanh \odot \sigma$	softmax	none	L1, L3
Dual-stream conjunctive	gate + critical query	blend of two streams	$\lambda \rightarrow 0$	L3

Table 6 averages each head over every encoder it was paired with. Two things here support statements in the main text. First, the spread caused by the head choice (0.744 to 0.921) is as large as the spread caused by the encoder choice in Table 4 (0.771 to 0.937), so the pooling head is a main design choice and not a detail. Second, being able to return to the baseline is *not* a safety guarantee: five of the seven heads contain conjunctive pooling as a special case, and the dual-stream head, which is one of them, is still the worst head we tested — its four pairings score 0.860, 0.824, 0.736 and 0.556. The reason is visible in the training runs: its mixing weight starts at $\lambda_0 = 0.05$ and nothing in Equation (6) pushes it back towards the baseline setting, so being *able* to return to the baseline does not mean training will move it there. That is why the main text argues for Top- k using measured NDCG@ n and not only this property.

Table 6: Heads: the mean over every encoder the head was paired with, WebTraffic. This leaves out the κ and λ_{focus} ablation runs and the published-recipe baseline, which would otherwise give extra weight to Top- k and to the bottleneck encoder.

Head	Acc.	AOPCR	NDCG@ n	n
Class-wise conjunctive	0.921	2.322	0.692	10
Per-class gated attention	0.918	1.869	0.733	4
Adaptive class-wise	0.912	2.210	0.732	9
Softmax conjunctive	0.907	1.690	0.712	9
Top- k conjunctive	0.904	2.267	0.712	16
Attention-max	0.838	1.758	0.645	8
Dual-stream conjunctive	0.744	2.007	0.636	4

B.3 HYPERPARAMETERS

The same for every model and every dataset unless stated: Adam (Kingma & Ba, 2015), learning rate 1.25×10^{-3} , weight decay 1.2×10^{-4} , at most 400 epochs, early stopping patience 60 on validation loss, batch size $\min(\lfloor n_{\text{train}}/10 \rfloor, 16)$ with a floor of 2, label smoothing 0.13, stratified 80/20 validation split when $n_{\text{train}} \geq 100$ and training-set monitoring otherwise. Encoder: $d = 64$, 4 blocks (wide: $d = 128$, 6 blocks), kernels $\{5, 11, 23\}$, dilation doubling per block capped at 16, dropout 0.2. Head: attention width 8, $\kappa = 0.1$, $\tau_0 = 1.0$, $\beta_0 = 0.3$, $\lambda_0 = 0.5$ (attention-max) or 0.05 (dual-stream). Loss: $\lambda_{\text{focus}} = 0.01$ for our models and 0 for the MILLET baselines, since the term is our addition. The published-recipe baseline uses MILLET’s own settings instead, where the difference that matters is 1,500 epochs against our 400. WebTraffic is produced at the released defaults: 1,000 series, length 1,008, 10 classes, 50/50 train/test split. Each series is standardised on its own; nothing else is done to the data.

C ADDITIONAL RESULTS

C.1 PER-SEED RESULTS

Table 7: The three models run with three seeds on WebTraffic. The standard deviations in the last three columns are the level of run-to-run noise used throughout the paper: we never report a difference smaller than the matching one as a result.

Model	seed 0	seed 1	seed 2	Acc. mean $\pm$ sd	AOPCR sd	NDCG@n sd
SEA-Net gated + Top- k	0.954	0.958	0.952	0.955 ± 0.003	± 0.131	± 0.022
SEA-Net bottleneck + Top- k	0.938	0.938	0.966	0.947 ± 0.016	± 0.214	± 0.013
MILLET (matched budget)	0.894	0.876	0.892	0.887 ± 0.010	± 0.884	± 0.016

The single-seed rows of Table 3 are all the bottleneck Top- k model with one part or one hyperparameter changed, so its spreads are the right noise level for them. Note that seed 0 of that model gives 0.938, not its three-seed mean of 0.947; Table 3 therefore uses 0.938, so that every cell in both of its panels comes from the same seed.

C.2 UCR ARCHIVE RESULTS

Table 8: UCR-85 accuracy and mean rank. Mean rank is over the 84 datasets finished by all 28 models we ran on the archive; lower is better. Win/tie/loss is counted per dataset against MILLET’s *published* accuracies. [‡]84 of 85 datasets finished.

Model	Params	Acc.	Mean rank	W/T/L
MILLET, published (Early et al., 2024)	423,707	0.8445	—	—
MILLET, published recipe, our code	423,707	0.8434 [‡]	9.41	26/32/26
SEA-Net wide + class-wise conjunctive	269,164	0.8292	11.53	26/19/39
SEA-Net wide + MILLET additive	269,083	0.8298	11.98	23/19/42
SEA-Net wide + MILLET conjunctive	269,083	0.8238	12.43	22/20/42
MILLET, matched budget	423,707	0.8274 [‡]	12.60	13/25/46
ResNet + conjunctive	506,331	0.8146	13.54	25/11/49
FCN + conjunctive	267,035	0.8141	14.48	20/12/52
SEA-Net gated + Top- k	61,740	0.8083	15.29	19/13/53
SEA-Net bottleneck + Top-k	41,324	0.8097	15.40	20/15/50

Table 8 is the evidence behind Section 5.3. Two comparisons are worth separating. Keeping the training budget fixed and changing the backbone at a similar size moves the mean rank from 12.60 to 11.53, about 1.1 places. Keeping the architecture fixed and changing only the training budget moves it from 12.60 to 9.41, about 3.2 places — three times as much. Any comparison of backbones in this field that does not keep the budget fixed is mostly measuring the budget.

A Wilcoxon–Holm test over the same datasets finds that most of the differences in the middle of this table are not significant at $\alpha = 0.05$. That is one more reason the main text argues from WebTraffic

and from cost rather than from UCR rank, and a reason we give the win/tie/loss counts above instead of relying on the rank order alone.

C.3 COMPARISON WITH PUBLISHED MILLET RESULTS

Table 9: Five ways of getting a MILLET number, and what each one is evidence for.

How the number was obtained	WebTraffic Acc.	UCR-85 Acc.
Published, mean of 5 seeds (Early et al., 2024)	0.924	0.8445
Published, 5-model ensemble	0.940	—
Released weights, evaluated in our pipeline	0.922	—
Re-trained here with <i>their</i> hyperparameters	0.920	0.8434
Re-trained here with <i>our</i> settings	0.887	0.8274

The third and fourth rows agree with the first to within 0.4 and 0.1 accuracy points, which is the check that our re-implementation is correct. The gap between the fourth and fifth rows is therefore the training budget and nothing else, and it is why the main comparison in Table 1 treats the matched-budget row as the like-for-like baseline while still showing the published-recipe row as the harder target. We do not copy MILLET’s published AOPCR into this table: Section 5.5 shows it could not be compared with any value we measured, and quoting it would go against our own finding.

C.4 ALL MODEL COMBINATIONS

Table 10 lists all 72 encoder–head combinations, ordered by WebTraffic accuracy. The UCR columns are blank where the combination was not run on the archive; those combinations were tested on WebTraffic only, because the compute was limited. One row, `seanet.bottleneck.adaptive`, finished only 6 of the 85 UCR datasets, so we leave its UCR values out instead of reporting them — a mean over 6 datasets cannot be compared with a mean over 85.

Table 10: All 72 encoder–head combinations, ordered by WebTraffic accuracy. A dash in the UCR column means the combination was tested on WebTraffic only. Pooling heads marked * are MILLET’s own, used unchanged.

#	Configuration	Encoder	Head	Acc.	AOPCR	NDCG@n	UCR-85	Params
1	<code>seanet.gated.mean.topk</code>	<code>gated</code>	<code>Top-k</code>	0.955	2.23	0.750	0.8083	61,740
2	<code>seanet.conjunctive</code>	<code>plain</code>	<code>conjunctive*</code>	0.954	1.50	0.698	0.8238	269,083
3	<code>seanet.gated.max.topk</code>	<code>gated</code>	<code>Top-k</code>	0.952	2.27	0.719	0.8108	61,740
4	<code>seanet.topk.nofocus</code>	<code>bottleneck</code>	<code>Top-k</code>	0.950	2.30	0.765	—	41,324
5	<code>seanet.spiketrend.topk</code>	<code>spike/trend</code>	<code>Top-k</code>	0.950	2.32	0.756	0.8089	67,020
6	<code>seanet.inputgate.mschan</code>	<code>ms-channels</code>	<code>Top-k</code>	0.948	2.32	0.757	—	69,768
7	<code>seanet.gated.mschan</code>	<code>ms-channels</code>	<code>Top-k</code>	0.948	2.22	0.717	—	67,592
8	<code>seanet.bottleneck.topk</code>	<code>bottleneck</code>	<code>Top-k</code>	0.947	2.62	0.772	0.8097	41,324
9	<code>seanet.recon.adaptive</code>	<code>recon</code>	<code>adaptive</code>	0.946	2.60	0.768	0.8140	62,935
10	<code>seanet.inputgate.topk</code>	<code>input-gated</code>	<code>Top-k</code>	0.946	2.80	0.748	0.8131	58,092
11	<code>seanet.classwise</code>	<code>plain</code>	<code>class-wise</code>	0.946	1.72	0.686	0.8292	269,164
12	<code>seanet.gated.max</code>	<code>gated</code>	<code>class-wise</code>	0.944	1.89	0.592	0.8237	61,740
13	<code>seanet.gated.last.topk</code>	<code>gated</code>	<code>Top-k</code>	0.942	2.31	0.748	0.8015	61,740
14	<code>seanet</code>	<code>plain</code>	<code>additive*</code>	0.942	1.58	0.729	0.8298	269,083
15	<code>seanet.gated.last</code>	<code>gated</code>	<code>class-wise</code>	0.942	2.52	0.742	0.7888	61,740
16	<code>seanet.recon.topk</code>	<code>recon</code>	<code>Top-k</code>	0.940	2.51	0.698	0.8063	62,925
17	<code>seanet.topk.k025</code>	<code>bottleneck</code>	<code>Top-k</code>	0.940	2.65	0.733	—	41,324
18	<code>seanet.bottleneck.softmax</code>	<code>bottleneck</code>	<code>softmax</code>	0.940	1.68	0.740	0.8106	41,325
19	<code>seanet.gated.last.adaptive</code>	<code>gated</code>	<code>adaptive</code>	0.938	2.39	0.793	0.8029	61,750
20	<code>seanet.spiketrend.adaptive</code>	<code>spike/trend</code>	<code>adaptive</code>	0.934	1.91	0.747	0.8010	67,030
21	<code>seanet.recon.softmax</code>	<code>recon</code>	<code>softmax</code>	0.932	1.64	0.723	—	62,926
22	<code>seanet.slim.topk</code>	<code>plain</code>	<code>Top-k</code>	0.932	2.69	0.778	0.8093	57,580
23	<code>seanet.bottleneck.gatedattn</code>	<code>bottleneck</code>	<code>gated-attn</code>	0.930	2.23	0.741	0.8146	41,844
24	<code>seanet.spiketrend.gatedattn</code>	<code>spike/trend</code>	<code>gated-attn</code>	0.930	1.59	0.721	0.8039	67,540
25	<code>seanet.gated</code>	<code>gated</code>	<code>class-wise</code>	0.930	2.41	0.637	0.7900	61,740
26	<code>seanet.slim.adaptive</code>	<code>plain</code>	<code>adaptive</code>	0.930	2.45	0.793	—	57,590
27	<code>seanet.bottleneck.mschan</code>	<code>ms-channels</code>	<code>Top-k</code>	0.926	2.24	0.777	—	47,176
28	<code>seanet.bottleneck.mschan.lean</code>	<code>ms-channels</code>	<code>Top-k</code>	0.924	2.76	0.735	—	43,576
29	<code>seanet.gated.last.softmax</code>	<code>gated</code>	<code>softmax</code>	0.920	1.99	0.738	—	61,741
30	<code>millet.paper</code>	<code>InceptionTime</code>	<code>conjunctive*</code>	0.920	13.27	0.661	0.8434	423,707
31	<code>seanet.acp</code>	<code>plain</code>	<code>adaptive</code>	0.918	1.61	0.619	0.8134	269,174

continued on the next page

Table 10 continued from the previous page.

#	Configuration	Encoder	Head	Acc.	AOPCR	NDCG@n	UCR-85	Params
32	seanet.gated.last.gatedattn	gated	gated-attn	0.918	1.35	0.757	0.8048	62,260
33	seanet.slim	plain	class-wise	0.916	2.48	0.719	—	57,580
34	seanet.gated.max.softmax	gated	softmax	0.914	1.88	0.776	—	61,741
35	seanet.inputgate	input-gated	class-wise	0.914	2.48	0.715	—	58,092
36	seanet.gated.mean.softmax	gated	softmax	0.912	2.09	0.765	—	61,741
37	seanet.recon	recon	class-wise	0.910	2.50	0.680	—	62,925
38	seanet.topk.k100	bottleneck	Top-k	0.908	2.44	0.722	—	41,324
39	seanet.gated.max.adaptive	gated	adaptive	0.906	2.07	0.729	—	61,750
40	seanet.inputgate.adaptive	input-gated	adaptive	0.905	2.65	0.732	0.8153	58,102
41	seanet.spiketrend	spike/trend	class-wise	0.904	2.15	0.699	—	67,020
42	seanet.bottleneck	bottleneck	class-wise	0.902	2.57	0.733	—	41,324
43	seanet.gated.mean	gated	class-wise	0.902	2.49	0.714	—	61,740
44	seanet.inputgate.softmax	input-gated	softmax	0.894	1.64	0.696	—	58,093
45	seanet.slim.gatedattn	plain	gated-attn	0.894	2.30	0.714	—	58,100
46	seanet.recon.attnmax	recon	attn-max	0.892	2.32	0.725	—	62,935
47	seanet.slim.softmax	plain	softmax	0.890	1.83	0.705	—	57,581
48	seanet.slim.gatedattn	gated	attention*	0.888	2.40	0.710	—	58,100
49	seanet.topk.k005	bottleneck	Top-k	0.888	2.29	0.715	—	41,324
50	millet	InceptionTime	conjunctive*	0.887	2.57	0.677	0.8274	423,707
51	seanet.gated.max.attnmax	gated	attn-max	0.886	1.12	0.575	—	61,750
52	seanet.topk.k050	bottleneck	Top-k	0.882	3.11	0.732	—	41,324
53	seanet.softmax	plain	softmax	0.880	1.02	0.605	0.8131	269,165
54	seanet.spiketrend.softmax	spike/trend	softmax	0.878	1.43	0.665	—	67,021
55	seanet.bottleneck.adaptive	bottleneck	adaptive	0.868	1.97	0.680	—	41,334
56	seanet.slim.dualstream	plain	dual-stream	0.860	2.40	0.745	—	58,101
57	seanet.gated.mean.adaptive	gated	adaptive	0.860	2.24	0.731	—	61,750
58	seanet.inputgate.attnmax	input-gated	attn-max	0.858	2.15	0.700	—	58,102
59	seanet.spiketrend.attnmax	spike/trend	attn-max	0.854	1.65	0.644	—	67,030
60	seanet.gated.last.attnmax	gated	attn-max	0.854	1.78	0.668	—	61,750
61	seanet.bottleneck.shallow	bottleneck	Top-k	0.846	2.82	0.624	—	21,548
62	seanet.gated.mean.attnmax	gated	attn-max	0.832	1.67	0.704	—	61,750
63	seanet.gated.pyramid	ms-pyramid	Top-k	0.832	0.98	0.591	—	62,781
64	seanet.bottleneck.dualstream	bottleneck	dual-stream	0.824	2.04	0.652	—	41,845
65	seanet.bottleneck.attnmax	bottleneck	attn-max	0.816	1.74	0.610	—	41,334
66	seanet.bottleneck.pyramid	ms-pyramid	Top-k	0.786	1.03	0.569	—	42,365
67	resnet	ResNet	conjunctive*	0.772	2.95	0.554	0.8146	506,331
68	fcn	FCN	conjunctive*	0.742	3.83	0.533	0.8141	267,035
69	seanet.gated.last.dualstream	gated	dual-stream	0.736	1.95	0.637	—	62,261
70	seanet.slim.attnmax	plain	attn-max	0.714	1.63	0.532	—	57,590
71	seanet.bottleneck.mschan.pyramid	ms-pyramid	Top-k	0.696	2.17	0.653	—	28,441
72	seanet.spiketrend.dualstream	spike/trend	dual-stream	0.556	1.64	0.510	—	67,541